

Name: L. Sanjay Kumar

Reg No: 192425226

Course Code: CSA0903

Course Name: Programming in JAVA

Question 1: Single Inheritance – Vehicle and Car

Scenario

A company has a **Vehicle** class with methods `startEngine()` and `stopEngine()`. A **Car** class extends **Vehicle** and adds a `playMusic()` method.

The company now wants every car to automatically log the engine start time whenever `startEngine()` is called, while still using the original engine-start functionality.

Aim

To implement **Single Inheritance** in Java by extending the Vehicle class into the Car class and demonstrating method overriding using `super`.

Problem Statement

Develop a Java program in which a Car class inherits from a Vehicle class. The Car class should override the `startEngine()` method to display the engine start time while still executing the original functionality of the parent class. The program should also demonstrate inheritance by accessing inherited methods along with the newly added `playMusic()` method.

Java Program

```
import java.time.LocalDateTime;

class Vehicle {
    void startEngine() {
        System.out.println("Engine Started");
    }
    void stopEngine() {
        System.out.println("Engine Stopped");
    }
}
```

```
class Car extends Vehicle {
    @Override
    void startEngine() {
        System.out.println("Engine Start Time: " +
            LocalDateTime.now());
        super.startEngine();
    }
    void playMusic() {
        System.out.println("Playing Music");
    }
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        car.startEngine();
        car.playMusic();
        car.stopEngine();
    }
}
```

Sample Output

Engine Start Time: 2026-07-17T15:20:10

Engine Started

Playing Music

Engine Stopped

1. How would you implement this using single inheritance?

Create a Vehicle class and extend it using the Car class. Override the startEngine() method to add logging functionality.

2. Should Car override startEngine()? Why?

Yes. Overriding allows the Car class to log the engine start time while preserving the original functionality.

3. How can the parent class implementation still be executed?

By calling super.startEngine() inside the overridden method.

4. What happens if Car does not override startEngine()?

The Car object will use the inherited startEngine() method from the Vehicle class, and the engine start time will not be logged.

Question 2: Multilevel Inheritance – Employee, Developer and SeniorDeveloper

Scenario

A software company models employees as follows:

- **Employee** → stores employee ID and name.
- **Developer** extends Employee → stores programming language.
- **SeniorDeveloper** extends Developer → stores team size and performs code reviews.

Now HR requests that every **SeniorDeveloper** should be able to access employee information and programming language details without duplicating code.

Aim

To implement **Multilevel Inheritance** in Java by allowing the SeniorDeveloper class to inherit properties and methods from both Developer and Employee.

Problem Statement

Develop a Java program to demonstrate multilevel inheritance. Create an Employee class to store employee details, a Developer class that inherits from Employee and stores the programming language, and a SeniorDeveloper class that inherits from Developer and stores the team size. The program should display all inherited information without duplicating code.

Java Program

```
class Employee {  
    int empId;  
    String name;  
    Employee(int empId, String name) {  
        this.empId = empId;  
        this.name = name;  
    }  
}
```

```
        void displayEmployee() {
            System.out.println("Employee ID : " + empId);
            System.out.println("Employee Name : " + name);
        }
    }

    class Developer extends Employee {
        String language;

        Developer(int empId, String name, String language) {
            super(empId, name);
            this.language = language;
        }

        void displayLanguage() {
            System.out.println("Programming Language : " + language);
        }
    }

    class SeniorDeveloper extends Developer {
        int teamSize;

        SeniorDeveloper(int empId, String name, String language, int
teamSize) {
            super(empId, name, language);
            this.teamSize = teamSize;
        }

        void codeReview() {
            System.out.println("Performing Code Review");
        }

        void displayDetails() {
            displayEmployee();
            displayLanguage();
            System.out.println("Team Size : " + teamSize);
        }
    }

    public class Main {
        public static void main(String[] args) {
```

```
        SeniorDeveloper sd = new SeniorDeveloper(101, "John", "Java",
8);
        sd.displayDetails();
        sd.codeReview();
    }
}
```

Sample Output

Employee ID : 101

Employee Name : John

Programming Language : Java

Team Size : 8

Performing Code Review

1. Why is multilevel inheritance suitable here?

It allows SeniorDeveloper to inherit employee details and programming language information without duplicating code.

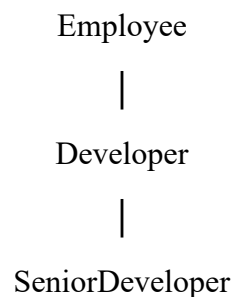
2. Which members are accessible in SeniorDeveloper?

Employee ID, employee name, programming language, employee methods, developer methods, and its own members.

3. If Developer overrides a method from Employee, which implementation will SeniorDeveloper inherit?

SeniorDeveloper inherits the overridden method from the Developer class.

4. Draw the inheritance hierarchy.



Question 3: Hierarchical Inheritance – Payment System

Scenario

An online shopping application has a base class **Payment**.

Three classes inherit from it:

- **CreditCardPayment**
- **UPIPayment**
- **NetBankingPayment**

Each payment type processes transactions differently but shares common validation logic.

Aim

To implement **Hierarchical Inheritance** in Java by creating multiple child classes that inherit common functionality from a single parent class.

Problem Statement

Develop a Java program to demonstrate hierarchical inheritance. Create a parent class named **Payment** containing common validation logic. Create three child classes namely **CreditCardPayment**, **UPIPayment**, and **NetBankingPayment**, each implementing its own transaction processing method while sharing the common validation method inherited from the parent class.

Java Program

```
class Payment {  
    void validate() {  
        System.out.println("Payment Validated Successfully");  
    }  
}  
  
class CreditCardPayment extends Payment {  
    void processTransaction() {  
        System.out.println("Credit Card Payment Processed");  
    }  
}  
  
class UPIPayment extends Payment {
```

```
        void processTransaction() {
            System.out.println("UPI Payment Processed");
        }
    }
    class NetBankingPayment extends Payment {
        void processTransaction() {
            System.out.println("Net Banking Payment Processed");
        }
    }
    public class Main {
        public static void main(String[] args) {
            CreditCardPayment cc = new CreditCardPayment();
            cc.validate();
            cc.processTransaction();
            System.out.println();
            UPIPayment upi = new UPIPayment();
            upi.validate();
            upi.processTransaction();
            System.out.println();
            NetBankingPayment nb = new NetBankingPayment();
            nb.validate();
            nb.processTransaction();
        }
    }
}
```

Sample Output

Payment Validated Successfully

Credit Card Payment Processed

Payment Validated Successfully

UPI Payment Processed

Payment Validated Successfully

Net Banking Payment Processed

1. Why is hierarchical inheritance appropriate here?

Multiple payment classes share the same validation logic while implementing different transaction processing methods.

2. Where should common validation logic be implemented?

In the parent Payment class.

3. How can each payment class provide its own transaction processing?

Each child class defines its own processTransaction() method.

4. What is the advantage over writing three independent classes?

It promotes code reuse, reduces duplication, simplifies maintenance, and makes the program easier to extend.

Question 4: Compile-Time Polymorphism (Method Overloading)

Scenario

A calculator application supports addition of:

- Two integers
- Three integers
- Two decimal numbers

The product owner wants users to call the same method name (add) regardless of the input.

Aim

To implement **Compile-Time Polymorphism (Method Overloading)** in Java by using the same method name with different parameter lists.

Problem Statement

Develop a Java program for a calculator application that performs addition using the same method name add(). The program should support adding two integers, three integers, and two decimal numbers. This demonstrates compile-time polymorphism through method overloading.

Java Program

```
class Calculator {
```



```
int add(int a, int b) {
    return a + b;
}

int add(int a, int b, int c) {
    return a + b + c;
}

double add(double a, double b) {
    return a + b;
}

}

public class Main {

    public static void main(String[] args) {

        Calculator c = new Calculator();

        System.out.println("Sum of 2 Integers : " + c.add(5, 10));
        System.out.println("Sum of 3 Integers : " + c.add(5, 10, 15));
        System.out.println("Sum of 2 Decimals : " + c.add(5.5, 10.5));
        System.out.println("Mixed Values : " + c.add(5, 10.0));
    }
}
```

Sample Output

Sum of 2 Integers : 15

Sum of 3 Integers : 30

Sum of 2 Decimals : 16.0

Mixed Values : 15.0

1. Which OOP concept should be used?

Method Overloading (Compile-Time Polymorphism).

2. How will Java decide which method to invoke?

Java selects the method based on the number, type, and order of parameters during compilation.

3. Can methods be overloaded by changing only the return type?

No. The parameter list must be different.

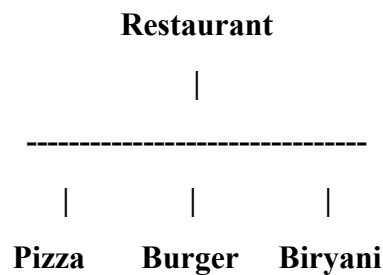
4. What happens if add(5, 10.0) is called?

Java automatically promotes 5 to double and invokes add(double, double).

Question 5: Runtime Polymorphism (Method Overriding)

Scenario

A food delivery application has:



Each restaurant calculates delivery time differently.

The application stores all restaurants in:

```
List<Restaurant> restaurants;
```

and calls:

```
restaurant.calculateDeliveryTime();
```

without checking the actual restaurant type.

Aim

To implement **Runtime Polymorphism (Method Overriding)** in Java by overriding a common method in different subclasses.

Problem Statement

Develop a Java program to demonstrate runtime polymorphism using method overriding.

Create a parent class **Restaurant** containing the method `calculateDeliveryTime()`. Create child

classes Pizza, Burger, and Biryani that override this method with their own delivery times. Store all objects using the parent class reference and invoke the method to demonstrate dynamic method dispatch.

Java Program

```
import java.util.ArrayList;
import java.util.List;
class Restaurant {
    void calculateDeliveryTime() {
        System.out.println("Default Delivery Time");
    }
}
class Pizza extends Restaurant {
    @Override
    void calculateDeliveryTime() {
        System.out.println("Pizza Delivery : 30 Minutes");
    }
}
class Burger extends Restaurant {
    @Override
    void calculateDeliveryTime() {
        System.out.println("Burger Delivery : 20 Minutes");
    }
}
class Biryani extends Restaurant {
    @Override
    void calculateDeliveryTime() {
        System.out.println("Biryani Delivery : 40 Minutes");
    }
}
public class Main {
    public static void main(String[] args) {
        List<Restaurant> restaurants = new ArrayList<>();
        restaurants.add(new Pizza());
```

```
        restaurants.add(new Burger());  
        restaurants.add(new Biryani());  
        for (Restaurant restaurant : restaurants) {  
            restaurant.calculateDeliveryTime();  
        }  
    }  
}
```

Sample Output

```
Pizza Delivery : 30 Minutes  
Burger Delivery : 20 Minutes  
Biryani Delivery : 40 Minutes
```

1. Which OOP concept is being used?

Runtime Polymorphism (Method Overriding).

2. Which version of calculateDeliveryTime() executes?

The overridden method of the actual object (Pizza, Burger, or Biryani) executes.

3. When is the method selection decided?

The method is selected at runtime based on the object's actual type.

4. What would happen if calculateDeliveryTime() were declared static?

Method overriding would not occur. The method call would depend on the reference type (Restaurant) instead of the actual object, resulting in compile-time binding.

Question 6: Interface – Smart Home Devices

Scenario

A smart home application supports different devices:

- **Smart Bulb**

- **Smart Fan**
- **Smart AC**

All devices can be turned on and turned off, but each device has its own implementation.

In the future, third-party manufacturers should also be able to integrate their own devices without modifying existing code.

Aim

To implement the **Interface** concept in Java by defining common operations for different smart home devices while allowing each device to provide its own implementation.

Problem Statement

Develop a Java program using an interface named SmartDevice that declares the methods turnOn() and turnOff(). Create classes SmartBulb, SmartFan, and SmartAC that implement this interface and provide their own implementations of these methods. The program should demonstrate how interfaces support flexibility, code reusability, and easy integration of new devices.

Java Program

```
interface SmartDevice {

    void turnOn();
    void turnOff();
}

class SmartBulb implements SmartDevice {
    public void turnOn() {
        System.out.println("Smart Bulb is ON");
    }
    public void turnOff() {
        System.out.println("Smart Bulb is OFF");
    }
}

class SmartFan implements SmartDevice {
    public void turnOn() {
        System.out.println("Smart Fan is ON");
    }
    public void turnOff() {
```

```
        System.out.println("Smart Fan is OFF");
    }
}
class SmartAC implements SmartDevice {
    public void turnOn() {
        System.out.println("Smart AC is ON");
    }
    public void turnOff() {
        System.out.println("Smart AC is OFF");
    }
}
public class Main {
    public static void main(String[] args) {
        SmartDevice bulb = new SmartBulb();
        SmartDevice fan = new SmartFan();
        SmartDevice ac = new SmartAC();
        bulb.turnOn();
        bulb.turnOff();
        System.out.println();
        fan.turnOn();
        fan.turnOff();
        System.out.println();
        ac.turnOn();
        ac.turnOff();
    }
}
```

Sample Output

Smart Bulb is ON

Smart Bulb is OFF

Smart Fan is ON

Smart Fan is OFF

Smart AC is ON

Smart AC is OFF

1. Would you use an interface or inheritance? Why?

Use an **interface** because all devices perform the same operations (turnOn() and turnOff()) but each device has its own implementation. It also allows third-party manufacturers to add new devices without modifying existing code.

2. Design an appropriate interface.

```
interface SmartDevice {  
    void turnOn();  
    void turnOff();  
}
```

3. Can one device implement multiple interfaces? Give an example.

Yes. A class can implement multiple interfaces.

Example:

```
interface SmartDevice {  
    void turnOn();  
    void turnOff();  
}  
  
interface WiFiEnabled {  
    void connectWiFi();  
}  
  
class SmartAC implements SmartDevice, WiFiEnabled {  
    public void turnOn() {  
        System.out.println("AC ON");  
    }  
}
```

```
    public void turnOff() {  
        System.out.println("AC OFF");  
    }  
    public void connectWiFi() {  
        System.out.println("WiFi Connected");  
    }  
}
```

4. Why is an interface better than creating a common base class in this scenario?

An interface provides flexibility, supports multiple interface implementation, promotes loose coupling, and allows new smart devices from different manufacturers to be integrated without changing the existing application.